

Simulasi Engagement Optimized Matchmaking (EOMM) Mobile Legends Menggunakan Algoritma Branch and Bound, Greedy, dan Genetic Algorithm pada Kasus Lose Streak

Bagas Nur Ardianto - 24060123140063

Departemen Informatika

Universitas Diponegoro

Semarang, Indonesia

bagasardianto@students.undip.ac.id

Abstract—Engagement Optimized Matchmaking (EOMM) adalah paradigma matchmaking modern yang digunakan oleh berbagai game online kompetitif, termasuk Mobile Legends: Bang Bang (MLBB), yang secara eksplisit bertujuan memaksimalkan retensi pemain jangka panjang melalui manipulasi probabilitas kemenangan. Pada kondisi lose streak, pemain berada dalam keadaan psikologis rentan yang berpotensi menyebabkan churn jika tidak ditangani secara sistemik oleh mesin matchmaking. Penelitian ini memodelkan proses pemilihan empat rekan tim terbaik dari antrian pemain sebagai persoalan optimasi kombinatorial dengan fungsi objektif memaksimalkan total Skill Rating (SR) rekan tim. Tiga algoritma diimplementasikan dan dibandingkan secara eksperimental: Algoritma Greedy, Branch and Bound (B&B), dan Genetic Algorithm (GA). Eksperimen dilakukan pada empat skenario ukuran antrian, yaitu $N = 50$, $N = 100$, $N = 500$, dan $N = 1000$. Hasil menunjukkan bahwa B&B secara konsisten menghasilkan solusi optimal namun dengan kompleksitas waktu eksponensial pada kasus terburuk, Greedy menghasilkan solusi sub-optimal dengan waktu konstan $O(N \log N)$, sedangkan GA memberikan keseimbangan antara kualitas solusi dan efisiensi komputasi sebagaimana pendekatan heuristik. Penelitian ini memberikan kontribusi analitis berupa perbandingan kuantitatif tiga paradigma algoritma pada domain matchmaking game.

Keywords—EOMM, matchmaking, lose streak, Branch and Bound, Greedy, Genetic Algorithm, optimasi kombinatorial, Mobile Legends.

I. PENDAHULUAN

Industri game digital global telah mengalami pertumbuhan yang eksponensial dalam satu dekade terakhir, dengan proyeksi pendapatan global melebihi 200 miliar dolar Amerika Serikat pada tahun 2024 [1]. Di antara berbagai genre yang mendominasi pasar, Multiplayer Online Battle Arena (MOBA) menempati posisi yang sangat strategis berkat kombinasi unik antara kompleksitas strategi, kerja sama tim, dan siklus permainan yang relatif singkat namun intensif. Mobile Legends: Bang Bang (MLBB), yang dikembangkan oleh Moonton dan dirilis pada tahun 2016, telah menjadi salah satu game MOBA mobile paling dominan di kawasan Asia Tenggara, dengan basis pemain aktif bulanan yang diperkirakan melampaui 100 juta pengguna [1]. Keberhasilan komersial game semacam ini sangat bergantung pada kemampuan sistem untuk mempertahankan keterlibatan pemain (player engagement) dalam jangka panjang, yang secara langsung berkorelasi dengan pendapatan dari model bisnis freemium yang mengandalkan pembelian dalam aplikasi (in-app purchases).

Sistem matchmaking merupakan infrastruktur inti yang menentukan kualitas pengalaman bermain (gameplay experience) dalam setiap game online kompetitif. Paradigma

matchmaking yang paling fundamental dan historis dikenal adalah Skill-Based Matchmaking (SBMM), yang berupaya mempertemukan pemain-pemain dengan tingkat kemampuan yang setara sehingga pertandingan berjalan secara kompetitif dan adil [2]. Ukuran kemampuan dalam SBMM biasanya direpresentasikan melalui sistem rating seperti Elo Rating, Glicko-2, atau varian proprietary yang dikembangkan oleh masing-masing studio. Secara matematis, SBMM berupaya meminimalkan selisih rata-rata rating antara kedua tim yang bertanding, sehingga menciptakan pertandingan yang secara teoritis bersifat balanced. Namun, seiring berkembangnya ilmu perilaku konsumen dalam ekosistem game (game analytics), telah terjadi pergeseran paradigma yang signifikan. Para peneliti dan praktisi industri mulai menyadari bahwa keadilan pertandingan (fairness) tidak selalu berkorelasi positif dengan kepuasan pemain dan retensi jangka panjang.

Konsep Engagement Optimized Matchmaking (EOMM) pertama kali diformulasikan secara akademis oleh Chen et al. pada tahun 2017 [3]. EOMM berargumen bahwa sistem matchmaking seharusnya tidak sekadar memaksimalkan keadilan pertandingan, melainkan secara aktif memanipulasi komposisi tim dan probabilitas kemenangan untuk mengoptimalkan fungsi retensi pemain secara jangka panjang. Secara empiris, EOMM mendasarkan dirinya pada observasi bahwa pemain menunjukkan pola churn (berhenti bermain) yang berbeda-beda tergantung pada sejarah pertandingan mereka. Seorang pemain yang baru saja mengalami serangkaian kekalahan berturut-turut — kondisi yang dalam komunitas game dikenal sebagai lose streak — memiliki probabilitas churn yang jauh lebih tinggi dibandingkan pemain yang baru saja memenangkan pertandingan atau yang memiliki rekam jejak menang-kalah yang seimbang [3]. Implikasinya adalah bahwa sistem matchmaking yang cerdas seharusnya dapat mendeteksi kondisi lose streak seorang pemain dan secara proaktif mengarahkan pemain tersebut ke dalam skenario pertandingan di mana probabilitas kemenangannya lebih tinggi, guna memulihkan kondisi psikologis pemain dan mencegah churn.

Dari perspektif komputasional, tantangan utama yang muncul dari kerangka kerja EOMM pada kasus lose streak adalah persoalan pemilihan rekan tim (teammate selection) yang optimal dari sebuah kumpulan pemain yang sedang menunggu dalam antrian. Jika diasumsikan terdapat N pemain dalam antrian, dan sistem harus memilih tepat 4 rekan tim bagi seorang pemain lose streak, maka jumlah kombinasi yang mungkin secara matematis adalah $C(N, 4) = N! / (4! \times (N-4)!)$. Untuk $N = 100$, nilai ini adalah 3.921.225 kombinasi; untuk $N = 1000$, nilainya membengkak menjadi lebih dari 41 miliar kombinasi. Evaluasi menyeluruh atas setiap kombinasi secara

brute force jelas tidak layak secara komputasional dalam konteks real-time matchmaking yang memiliki batasan latensi ketat. Persoalan ini secara struktural analog dengan persoalan 0/1 Knapsack — yakni memilih subset dari sekumpulan elemen (pemain) yang memaksimalkan nilai total (Skill Rating) tanpa melanggar kendala kapasitas (jumlah rekan tim = 4) — sehingga tergolong sebagai persoalan optimasi kombinatorial NP-Hard yang membutuhkan pendekatan algoritmik yang cerdas dan efisien [6, 7]. Penelitian ini hadir sebagai respons terhadap kesenjangan tersebut dengan mengimplementasikan dan membandingkan tiga strategi algoritmik yang berbeda dalam paradigma dan karakteristiknya: Greedy, Branch and Bound, dan Genetic Algorithm.

II. BATASAN MASALAH

Guna menjaga fokus dan validitas eksperimen, penelitian ini memberlakukan batasan-batasan sebagai berikut:

1. Ukuran antrian pemain (N) yang diujikan dibatasi pada empat nilai diskrit, yaitu $N = 50$, $N = 100$, $N = 500$, dan $N = 1000$. Data pemain dalam antrian merupakan data fiktif yang dibangkitkan secara pseudo-random untuk keperluan simulasi.
2. Setiap pemain dalam antrian direpresentasikan hanya oleh dua atribut numerik, yaitu Skill Rating (SR) yang berskala 100 hingga 1000, dan Win Rate (WR) yang berskala 30% hingga 70%. Faktor-faktor lain seperti komunikasi, koneksi internet, dan latensi tidak dimodelkan.
3. Sebuah tim dalam simulasi ini terdiri dari tepat 5 pemain, yaitu 1 pemain lose streak (subjek utama / user) ditambah tepat 4 rekan tim yang dipilih oleh algoritma dari antrian. Proses optimasi berfokus pada pemilihan 4 rekan tersebut.
4. Simulasi ini tidak memodelkan mekanisme party atau premade group. Seluruh pemain dalam antrian diasumsikan berstatus solo queue, sehingga setiap pemain adalah entitas independen yang dapat dipilih secara bebas.
5. Faktor role hero (seperti Marksman, Tank, Mage, Fighter, Support) tidak dipertimbangkan dalam fungsi objektif dan pemilihan rekan tim. Model optimasi semata-mata didasarkan pada nilai SR dan WR pemain.

III. DASAR TEORI

A. Konsep Dasar Engagement Optimized Matchmaking (EOMM)

Engagement Optimized Matchmaking (EOMM) pertama kali diperkenalkan oleh Chen et al. [3] sebagai alternatif formal terhadap pendekatan SBMM konvensional. Fondasi teoritis EOMM bersumber pada ilmu ekonomi perilaku (behavioral economics) dan psikologi reinforcement, yang menunjukkan bahwa manusia cenderung lebih termotivasi untuk terus berpartisipasi dalam sebuah aktivitas ketika rasio hadiah-hukuman dikelola secara strategis [10]. Dalam konteks game kompetitif, "hadiah" dapat diinterpretasikan sebagai kemenangan (win) dan "hukuman" sebagai kekalahan (loss). EOMM berargumen bahwa fungsi retensi pemain (player retention function) tidak linear terhadap win rate kumulatif, melainkan memiliki titik puncak (peak) pada rentang win rate

tertentu yang secara psikologis memberikan sensasi "hampir menang" (near-win effect) dan "harapan untuk menang" (winning expectation).

Secara formal, Chen et al. [3] mendefinisikan fungsi retensi pemain sebagai $R(p)$, di mana p adalah probabilitas kemenangan pemain dalam sebuah pertandingan. Fungsi ini diasumsikan berbentuk konkaf (concave) dengan satu puncak global pada nilai p^* yang tidak selalu berada di $p = 0.5$ (kondisi perfectly balanced). Persamaan fungsi retensi yang digunakan adalah:

$$R(p) = \alpha \times p \times (1 - p) + \beta \times p + \gamma$$

di mana α , β , dan γ adalah parameter yang dikalibrasi menggunakan data historis perilaku pemain, dan p merepresentasikan probabilitas kemenangan yang ditentukan oleh komposisi SR kedua tim [3]. Tujuan dari EOMM adalah memilih komposisi pertandingan sedemikian rupa sehingga nilai $R(p)$ dimaksimalkan untuk setiap pemain dalam pertandingan, dengan kendala bahwa setiap pemain harus ditempatkan dalam satu dan hanya satu tim. Dalam konteks lose streak, nilai p^* bagi pemain yang sedang dalam kondisi tersebut cenderung lebih tinggi dari $p = 0.5$, artinya sistem matchmaking secara sengaja akan mencarikan lawan yang lebih lemah dan rekan tim yang lebih kuat guna mendorong probabilitas kemenangan lebih tinggi, yang pada akhirnya memaksimalkan retensi [3].

Penelitian terkait kerangka kerja EOMM [3] menggarisbawahi pentingnya riwayat historis pemain (seperti frekuensi kekalahan berturut-turut) sebagai faktor penentu dalam fungsi utilitas matchmaking. Secara statistik, pemain yang tidak memenangkan pertandingan dalam periode waktu yang panjang memiliki probabilitas churn (berhenti bermain) yang teramplifikasi secara eksponensial. Temuan ini memperkuat justifikasi ilmiah mengapa sistem matchmaking perlu secara aktif mengintervensi kondisi lose streak dengan pemilihan rekan tim yang lebih superior.

B. Algoritma Greedy

Algoritma Greedy adalah paradigma algoritma yang membangun solusi secara inkremental dengan selalu memilih pilihan lokal yang tampak paling optimal pada setiap langkah, tanpa mempertimbangkan konsekuensi jangka panjang dari pilihan tersebut [6]. Nama "Greedy" (rakus) mencerminkan sifat dasarnya: algoritma ini tidak pernah membatalkan pilihan yang telah dibuat, tidak pernah mempertimbangkan ulang alternatif yang telah diabaikan, dan tidak melakukan pencarian mundur (backtracking). Kekuatan utama algoritma Greedy adalah kesederhanaan implementasi dan efisiensi komputasional yang sangat tinggi — umumnya berorde $O(N \log N)$ untuk kasus yang memerlukan pengurutan.

Secara formal, sebuah algoritma Greedy terdiri dari tiga komponen utama [6, 7]:

1. Fungsi Seleksi (Selection Function): Menentukan elemen kandidat mana yang paling "menjanjikan" untuk dimasukkan ke dalam solusi. Dalam konteks penelitian ini, fungsi seleksi adalah pemilihan pemain dengan nilai SR tertinggi dari antrian yang belum dipilih.
2. Fungsi Kelayakan (Feasibility Function): Memeriksa apakah penambahan elemen yang dipilih ke dalam solusi parsial saat ini tidak akan melanggar kendala

yang ditetapkan. Dalam konteks ini, kendala adalah bahwa jumlah rekan tim yang telah dipilih belum mencapai 4.

3. Fungsi Tujuan (Objective Function): Fungsi yang ingin dimaksimalkan atau diminimalkan. Dalam konteks ini, fungsi tujuan adalah memaksimalkan total SR seluruh rekan tim yang terpilih.

Properti penting yang harus dipenuhi agar algoritma Greedy menghasilkan solusi global optimal adalah Greedy-Choice Property dan Optimal Substructure Property [6]. Greedy-Choice Property menyatakan bahwa pilihan lokal terbaik selalu merupakan bagian dari solusi global optimal, sedangkan Optimal Substructure Property menyatakan bahwa solusi optimal dari suatu masalah mengandung solusi optimal dari sub-masalahnya. Pada persoalan pemilihan rekan tim berdasarkan SR tertinggi dengan kendala jumlah, properti Greedy-Choice terpenuhi secara langsung karena pemain dengan SR tertinggi secara matematis selalu memberikan kontribusi terbesar terhadap total SR. Oleh karena itu, pada versi sederhana persoalan ini (hanya mempertimbangkan SR), Greedy akan menghasilkan solusi yang sama dengan solusi optimal. Namun, ketika fungsi fitness diperluas untuk mencakup kombinasi SR dan WR secara berbobot, sifat ini dapat tidak terpenuhi, sehingga Greedy berpotensi menghasilkan solusi sub-optimal [7].

C. Algoritma Branch and Bound (B&B)

Branch and Bound (B&B) adalah teknik algoritma yang digunakan untuk menyelesaikan persoalan optimasi kombinatorial secara eksak (exact optimization), artinya algoritma ini menjamin bahwa solusi yang dihasilkan adalah solusi optimal global [6, 7]. B&B merupakan penyempurnaan signifikan dari algoritma Backtracking karena mengintegrasikan mekanisme pemangkasan (pruning) berbasis perhitungan batas atas (upper bound) atau batas bawah (lower bound) yang memungkinkan pengabaian seluruh cabang pohon pencarian yang terbukti tidak dapat menghasilkan solusi yang lebih baik dari solusi terbaik yang telah ditemukan sejauh ini.

Struktur data yang mendasari B&B adalah State Space Tree (pohon ruang status), yaitu pohon imajiner yang setiap simpulnya (node) merepresentasikan sebuah keputusan parsial dalam proses pembangunan solusi [7]. Dalam konteks penelitian ini, setiap simpul di kedalaman k pada pohon merepresentasikan kondisi di mana k pemain telah dipilih (atau secara eksplisit tidak dipilih) dari antrian. Simpul akar merepresentasikan kondisi awal di mana belum ada pemain yang dipilih, dan simpul-simpul daun (leaf nodes) merepresentasikan kondisi di mana tepat 4 pemain telah dipilih — yang merupakan kandidat solusi valid.

Komponen kunci dalam B&B adalah Bounding Function, yaitu suatu fungsi yang menghitung batas atas (upper bound) dari nilai total SR terbaik yang mungkin dapat dicapai oleh seluruh simpul yang merupakan keturunan dari simpul saat ini [6]. Jika batas atas dari suatu simpul terbukti tidak melampaui nilai solusi terbaik yang telah ditemukan sebelumnya (best-so-far), maka seluruh subpohon yang berakar pada simpul tersebut dapat dipangkas (pruned) tanpa perlu dieksplorasi lebih lanjut. Mekanisme ini adalah kunci efisiensi B&B dibandingkan Brute Force. Secara formal, proses B&B pada penelitian ini bekerja sebagai berikut. Pertama, antrian pemain diurutkan secara menurun berdasarkan SR. Kedua, untuk

setiap simpul yang sedang dieksplorasi pada kedalaman k (artinya telah memilih k pemain dari indeks 0 hingga $i-1$), dihitung batas atasnya:

$$UB(node) = SR_{\text{terpilih saat ini}} + \sum SR[j], \text{ untuk } k \text{ pemain teratas yang tersisa dengan indeks } j \geq i$$

di mana $SR_{\text{terpilih saat ini}}$ adalah total SR dari k pemain yang telah dipilih pada simpul saat ini, dan penjumlahan dilakukan atas $(4 - k)$ pemain dengan SR tertinggi yang tersisa dalam antrian yang belum dipertimbangkan. Jika $UB(node) \leq Best_SR$ (nilai total SR terbaik yang telah ditemukan), maka simpul ini dan seluruh keturunannya dipangkas (pruned). Kondisi pemangkasan secara formal dinyatakan:

$$\text{Jika } UB(node) \leq Best_SR, \text{ maka lakukan PRUNING pada simpul tersebut.}$$

Pemangkasan ini valid dan tidak menyebabkan hilangnya solusi optimal karena: (1) antrian telah diurutkan menurun berdasarkan SR, sehingga batas atas yang dihitung merupakan batas atas yang paling ketat (tightest upper bound) yang mungkin; (2) tidak ada kombinasi elemen yang tersisa yang dapat menghasilkan total SR melebihi $UB(node)$; (3) oleh karena itu, tidak ada solusi yang lebih baik yang mungkin ditemukan di bawah simpul ini [7]. Kompleksitas waktu B&B pada kasus terburuk tetap eksponensial $O(2^N)$ namun dalam praktik jauh lebih cepat dari Brute Force karena efektivitas pruning yang tinggi [6].

D. Genetic Algorithm (GA)

Genetic Algorithm (GA) adalah metode pencarian dan optimasi metaheuristik yang terinspirasi dari mekanisme evolusi biologis darwinian, khususnya konsep seleksi alam, pewarisan sifat, dan mutasi genetik [8]. GA tergolong sebagai algoritma heuristik berbasis populasi (population-based heuristic), artinya algoritma ini tidak mencari satu solusi pada satu waktu, melainkan memelihara dan mengembangkan sekumpulan solusi kandidat (yang disebut populasi) secara simultan dari satu generasi ke generasi berikutnya. GA tidak menjamin ditemukannya solusi optimal global, namun secara empiris mampu menghasilkan solusi yang sangat baik (near-optimal) pada persoalan optimasi kombinatorial berukuran besar dalam waktu yang jauh lebih singkat dibandingkan metode pencarian eksak [8, 9].

Komponen-komponen utama GA dalam konteks penelitian ini adalah sebagai berikut:

1. Representasi Kromosom: Setiap kromosom merepresentasikan satu kandidat solusi, yaitu satu subset berisi 4 indeks pemain yang dipilih dari antrian. Kromosom direpresentasikan sebagai array integer berisi 4 elemen, misalnya [3, 17, 42, 89] yang berarti pemain dengan indeks ke-3, ke-17, ke-42, dan ke-89 dipilih sebagai rekan tim [8].
2. Fungsi Fitness: Setiap kromosom dievaluasi dengan fungsi fitness $f(\text{kromosom})$ yang merupakan fungsi objektif penelitian ini. Pada konfigurasi dasar, $f(\text{kromosom}) = SR[i] + SR[j] + SR[k] + SR[l]$, di mana i, j, k, l adalah indeks pemain yang dikodekan dalam kromosom. Kromosom yang encode indeks tidak valid atau duplikat diberi penalti fitness = 0 [9].
3. Seleksi: Digunakan metode Tournament Selection, di mana k kromosom dipilih secara random dari populasi dan kromosom dengan fitness tertinggi di

antara k tersebut menjadi induk (parent). Metode ini memiliki tekanan selektif yang dapat dikontrol melalui parameter k [8].

4. Crossover (Rekombinasi): Dua kromosom induk disilangkan untuk menghasilkan kromosom anak. Dalam konteks persoalan subset, digunakan metode Order Crossover (OX) atau uniform crossover yang dimodifikasi untuk memastikan tidak ada duplikasi indeks dalam kromosom anak [9].
5. Mutasi: Dengan probabilitas mutasi p_m yang kecil (tipikal 0.01 hingga 0.05), salah satu elemen (indeks pemain) dalam kromosom diganti secara random dengan indeks pemain lain yang belum terdapat dalam kromosom tersebut. Mutasi berfungsi menjaga keragaman genetik populasi dan mencegah konvergensi prematur ke local optimum [8].

GA beriterasi selama sejumlah generasi G atau hingga kriteria konvergensi terpenuhi (misalnya tidak ada perbaikan signifikan pada fitness terbaik selama T generasi terakhir). Kompleksitas per generasi adalah $O(|P| \times N)$, di mana $|P|$ adalah ukuran populasi dan N adalah ukuran antrian pemain [9].

IV. METODOLOGI

A. Pemodelan Masalah dan Struktur Data Antrian

Pada penelitian ini, antrian pemain dimodelkan sebagai sebuah larik (array) terurut yang menyimpan N objek pemain. Setiap objek pemain memiliki tiga atribut, yaitu: (1) ID, berupa pengidentifikasi unik berbentuk string (misalnya "P001", "P002", ..., "P1000"); (2) Skill Rating (SR), berupa bilangan bulat pada rentang [100, 1000] yang dibangkitkan secara pseudo-random menggunakan distribusi uniform; dan (3) Win Rate (WR), berupa bilangan riil pada rentang [0.30, 0.70] yang dibangkitkan secara pseudo-random menggunakan distribusi uniform. Pemain lose streak — yakni pemain yang menjadi subjek utama matchmaking — tidak dimasukkan ke dalam antrian dan tidak dapat dipilih sebagai rekan timnya sendiri. Pemain lose streak ini diasumsikan memiliki $SR = 450$ dan $WR = 0.38$, yang merepresentasikan kondisi seorang pemain rating menengah dengan win rate yang sedang menurun akibat serangkaian kekalahan.

Struktur data antrian secara konseptual direpresentasikan sebagai berikut. Misalkan Q adalah antrian berisi N pemain, maka:

$$Q = \{p_1, p_2, p_3, \dots, p_N\}$$

$$p_i = (ID_i, SR_i, WR_i), \text{ di mana } SR_i \in [100, 1000] \text{ dan } WR_i \in [0.30, 0.70]$$

Data pemain fiktif dibangkitkan secara programatik untuk setiap skenario eksperimen ($N = 50, 100, 500, 1000$). Guna memastikan reproducibility eksperimen, setiap bangkitan data menggunakan nilai seed generator bilangan pseudo-random yang tetap. Tabel 3.1 menampilkan contoh 10 baris pertama dari data antrian yang dibangkitkan untuk skenario $N = 50$ sebagai ilustrasi.

Tabel 4.1 Contoh Data Antrian Pemain Fiktif (10 Pemain Pertama, $N = 50$)

ID Pemain	Skill Rating (SR)	Win Rate (WR)	Keterangan
P001	872	0.61	Pemain Kuat

P002	341	0.44	Pemain menengah bawa
P003	955	0.68	Pemain sangat kuat
P004	519	0.52	Pemain menengah
P005	203	0.35	Pemain lemah
P006	788	0.57	Pemain kuat
P007	634	0.49	Pemain menengah atas
P008	142	0.31	Pemain sangat lemah
P009	901	0.65	Pemain sangat kuat
P010	467	0.48	Pemain menengah

B. Fungsi Objektif dan Kendala Optimasi

Persoalan pemilihan rekan tim ini diformulasikan secara matematis sebagai persoalan optimasi kombinatorial dengan satu fungsi tujuan dan satu kendala utama. Misalkan $S \subseteq Q$ adalah subset pemain yang dipilih sebagai rekan tim, dan setiap pemain dipresentasikan oleh variabel keputusan biner $x_i \in \{0, 1\}$, di mana $x_i = 1$ jika pemain ke- i dipilih dan $x_i = 0$ jika tidak.

Fungsi objektif yang ingin dimaksimalkan adalah total Skill Rating gabungan (SR kumulatif) dari seluruh rekan tim yang terpilih. Fungsi objektif ini memodelkan asumsi EOMM bahwa menempatkan seorang pemain lose streak bersama rekan-rekan dengan SR tinggi akan meningkatkan probabilitas kemenangan timnya, sesuai dengan model komputasional skill-based win probability [3]:

$$\text{Maksimalisasi: } f(s) = \sum_{i=1}^N x_i \times SR_i$$

Fungsi ini dikenai kendala-kendala berikut:

$$\text{Kendala 1 (Kapasitas Tim): } f(s) = \sum_{i=1}^N x_i \times SR_i = 4$$

$$\text{Kendala 2 (Biner): } x_i \in \{0, 1\}, \forall i \in \{1, 2, \dots, N\}$$

$$\text{Kendala 3 (Keseimbangan Tim): } |\mu_{\{SR, Tim\}} - \mu_{\{SR, Musuh\}}| \leq \Delta$$

Kendala 1 memastikan bahwa tepat 4 rekan tim dipilih. Kendala 2 mengharuskan setiap pemain dipilih atau tidak dipilih (tidak ada pemilihan parsial). Kendala 3 adalah kendala keseimbangan opsional yang memastikan bahwa rata-rata SR tim subjek tidak melebihi rata-rata SR tim lawan lebih dari batas toleransi Δ (dalam eksperimen ini, $\Delta = 200$), guna mencegah ketimpangan pertandingan yang ekstrem. Untuk penyederhanaan pada eksperimen utama, Kendala 3 direlaksasikan dan analisisnya disajikan secara terpisah sebagai eksperimen sekunder.

Fungsi objektif gabungan yang juga mempertimbangkan Win Rate dirumuskan sebagai fungsi Komposit Score (CS) dengan pembobotan:

$$CS(S) = w_1 \times (\sum_{i \in S} SR_i) + w_2 \times (\sum_{i \in S} WR_i \times 1000)$$

di mana w_1 dan w_2 adalah bobot pembobotan dengan $w_1 + w_2 = 1$. Dalam konfigurasi eksperimen primer, digunakan $w_1 = 0.7$ dan $w_2 = 0.3$, sehingga SR mendominasi fungsi evaluasi namun WR tetap diperhitungkan. Faktor perkalian 1000 pada suku WR digunakan untuk menormalisasi WR (yang berskala [0.30, 0.70]) ke rentang yang sebanding dengan SR (yang berskala [100, 1000]).

C. Desain Fungsi Pembatas (Bounding Function) Branch and Bound

Keberhasilan dan efisiensi Branch and Bound sangat bergantung pada kualitas bounding function yang digunakan. Bounding function yang baik harus memenuhi dua syarat: (1) selalu menghasilkan nilai yang valid sebagai batas atas (tidak pernah underestimate nilai optimal yang sesungguhnya), dan (2) seakurat mungkin (tidak terlalu overestimate) agar pemangkasan dapat terjadi sesering mungkin [7].

Pada penelitian ini, bounding function dirancang dengan prosedur sebagai berikut. Sebelum proses pencarian dimulai, antrian pemain Q diurutkan secara menurun berdasarkan SR, menghasilkan urutan $Q' = (q_1, q_2, \dots, q_N)$ di mana $SR(q_1) \geq SR(q_2) \geq \dots \geq SR(q_N)$. Misalkan pada suatu simpul pohon yang mewakili kedalaman k (yakni telah mengambil keputusan pilih/tidak-pilih terhadap k elemen pertama antrian terurut), dan telah terpilih m buah pemain dengan total SR sementara sebagai SR_{cur} :

$$UB(k, m, SR_{cur}) = SR_{cur} + \sum_{j=k+1}^N \{k + (4-m)\} SR(q_j)$$

Rumus ini menyatakan bahwa batas atas dari simpul saat ini adalah total SR yang sudah terpilih (SR_{cur}) ditambah SR dari $(4 - m)$ pemain berikutnya dalam antrian terurut yang belum diputuskan. Penjumlahan ini merupakan batas atas yang valid karena pemain-pemain tersebut adalah yang memiliki SR tertinggi di antara yang belum diputuskan, dan kita selalu dapat "memilih" mereka untuk tujuan kalkulasi batas atas. Kondisi pemangkasan yang diterapkan secara formal adalah:

Jika $UB(k, m, SR_{cur}) \leq Best_CS_so_far$, maka simpul ini DIPANGKAS (pruned).

Validitas kondisi pemangkasan ini dapat dibuktikan secara induktif: karena antrian terurut menurun dan UB mengambil elemen-elemen terbaik yang tersisa, maka tidak ada simpul anak dari simpul ini yang dapat menghasilkan total SR (atau CS) yang melebihi UB. Oleh karena itu, jika UB sendiri sudah tidak melampaui $Best_CS_so_far$, maka eksplorasi lebih lanjut pada subpohon ini dijamin tidak akan memperbarui solusi terbaik [7]. Efektivitas pemangkasan ini secara empiris meningkat seiring bertambahnya N karena lebih banyak simpul yang dapat dipangkas lebih awal, meskipun kompleksitas kasus terburuk tetap eksponensial.

D. Skenario dan Metrik Pengujian

Eksperimen dirancang untuk mengukur perbandingan performa ketiga algoritma secara sistematis dan terukur. Setiap algoritma diuji pada empat skenario ukuran antrian yang berbeda. Tabel 3.2 merangkum parameter eksperimen yang digunakan.

Tabel 4.2 Parameter Eksperimen untuk Setiap Skenario Pengujian

Skenario	N (Ukuran Antrian)	Metode Optimum Global	Parameter GA	Ulangan Eksperimen
S1	50	Exhaustive Search	$ P =50$, $G=200$, $p_m=0.02$	10 kali
S2	100	Exhaustive Search	$ P =100$, $G=300$, $p_m=0.02$	10 kali
S3	500	B&B (referensi)	$ P =200$, $G=500$, $p_m=0.03$	5 kali
S4	1000	B&B (referensi)	$ P =300$, $G=800$, $p_m=0.03$	5 kali

Dua metrik utama digunakan untuk mengevaluasi dan membandingkan performa ketiga algoritma:

1. Running Time (RT): Waktu eksekusi algoritma diukur dalam satuan milidetik (ms) menggunakan fungsi timer sistem dengan resolusi tinggi (`performance.now()` atau `time.perf_counter()`). Pengukuran dilakukan dari saat algoritma mulai memproses antrian hingga solusi akhir dikembalikan. Rata-rata dari seluruh ulangan eksperimen diambil sebagai nilai representatif.
2. Nilai Optimality (OPT%): Persentase kedekatan solusi yang ditemukan algoritma terhadap nilai optimum global, dihitung sebagai:

$$OPT\% = (CS_{algoritma} / CS_{optimal}) \times 100\%$$

di mana $CS_{algoritma}$ adalah nilai Composite Score yang ditemukan oleh algoritma yang bersangkutan, dan $CS_{optimal}$ adalah nilai Composite Score tertinggi yang dapat diperoleh (dari Exhaustive Search untuk S1-S2, dan dari B&B untuk S3-S4). Nilai $OPT\% = 100\%$ berarti algoritma menemukan solusi yang sama optimalnya dengan solusi terbaik yang diketahui. Semakin tinggi $OPT\%$ suatu algoritma, semakin baik kualitas solusinya. Nilai $OPT\%$ dari B&B selalu 100% karena algoritma ini bersifat eksak.

V. HASIL DAN PEMBAHASAN

A. Implementasi Algoritma dalam Lingkungan Simulasi Python

Seluruh eksperimen komputasional yang dijelaskan dalam makalah ini diimplementasikan menggunakan bahasa pemrograman Python versi 3.11.4, yang dipilih berdasarkan pertimbangan kemampuan komputasi numeriknya yang mumpuni, ketersediaan pustaka ilmiah seperti NumPy, SciPy, dan Matplotlib, serta kemudahan dalam melakukan prototyping algoritma yang bersifat iteratif maupun rekursif. Lingkungan eksperimen berjalan di atas sistem operasi Ubuntu 22.04 LTS dengan prosesor Intel Core i7-12700H berkecepatan 2.30 GHz dan kapasitas memori RAM sebesar 16 GB DDR5, sehingga memastikan konsistensi pengukuran waktu yang dapat diandalkan lintas skenario pengujian.

Setiap algoritma diimplementasikan sebagai modul Python yang independen dan terenkapsulasi: (1) modul `greedy_selector.py` mengimplementasikan Algoritma Greedy berbasis pengurutan; (2) modul `bnb_selector.py` mengimplementasikan Branch and Bound dengan State Space Tree rekursif; dan (3) modul `ga_selector.py` mengimplementasikan Genetic Algorithm berbasis populasi dengan operator Tournament Selection, Order Crossover, dan Bit-flip Mutation. Modul keempat, `data_generator.py`, bertugas membangkitkan data antrian pemain fiktif secara reproducible menggunakan seed tetap (`random.seed(42)`), sementara modul `benchmark.py` mengintegrasikan seluruh komponen untuk menjalankan skenario eksperimen secara otomatis dan mencatat hasil ke dalam berkas CSV terstruktur.

Pengukuran waktu eksekusi dilakukan menggunakan fungsi `time.perf_counter()` dari pustaka standar Python, yang menyediakan resolusi timer sub-milidetik pada sistem operasi berbasis POSIX. Setiap skenario dijalankan sebanyak 10 kali ulangan (untuk N = 50 dan N = 100) dan 5 kali ulangan (untuk N = 500 dan N = 1000), dan nilai waktu rata-rata dari seluruh ulangan digunakan sebagai nilai representatif yang dilaporkan dalam Tabel 4.1. Pendekatan ulangan ini bertujuan meredam variabilitas waktu yang disebabkan oleh fluktuasi sistem

operasi, seperti penjadwalan proses (process scheduling) dan cache miss.

Sebagai upaya menjamin transparansi dan reproducibility ilmiah, seluruh kode program eksperimen dalam penelitian ini diimplementasikan secara modular, terdokumentasi secara menyeluruh menggunakan docstring standar PEP 257, dan dapat diakses secara publik melalui repositori GitHub yang dapat diakses pada tautan berikut:

Repositori GitHub Simulasi EOMM: Repositori GitHub Simulasi EOMM: <https://github.com/bgsnr/eomm-mobile-legends>

Repositori tersebut mencakup seluruh modul algoritma, skrip pembangkit data, skrip benchmark, dan instruksi penggunaan yang terperinci dalam berkas README.md, sehingga memungkinkan pihak lain untuk mereplikasi, memverifikasi, dan mengembangkan eksperimen ini secara independen.

B. Analisis Waktu Komputasi (Running Time)

Dimensi pertama dalam evaluasi performa algoritma adalah efisiensi waktu komputasi, yang diukur sebagai waktu eksekusi aktual dalam satuan milidetik (ms). Tabel 4.1 di bawah ini menyajikan perbandingan running time rata-rata ketiga algoritma untuk keempat skenario ukuran antrian yang diujikan.

Tabel 5.1 Perbandingan Running Time Ketiga Algoritma (dalam Milidetik)

Ukuran Antrian (N)	Greedy (ms)	Branch & Bound (ms)	Genetic Algorithm (ms)
N = 50	1,2	3,8	28,4
N = 100	2,7	11,3	61,7
N = 500	14,1	4,820	512,9
N = 1000	31,6	84,730	1.247,3

Greedy tumbuh linier-logaritmik ($1,2 \rightarrow 31,6$ ms) sesuai kompleksitas $O(N \log N)$, dengan bottleneck tunggal pada operasi pengurutan TimSort. Branch and Bound menunjukkan lonjakan eksponensial — dari 3,8 ms (N=50) ke 84.730 ms (N=1000) — karena ruang pencarian teoritis mencapai $O(C(N,4)) \approx 41,6$ miliar kombinasi untuk N=1000. Meskipun demikian, mekanisme pruning berhasil memangkas rata-rata 99,9997% dari ruang pencarian, sehingga B&B masih jauh lebih efisien dari Brute Force murni. Genetic Algorithm tumbuh kuasi-linier ($28,4 \rightarrow 1.247,3$ ms) seiring peningkatan parameter $|P|$ dan G ; komponen terlambat adalah evaluasi fitness $O(|P| \times G)$ per skenario.

C. Analisis Kualitas Solusi (Optimality)

Tabel 5.2 Perbandingan Kualitas Solusi (Composite Score dan OPT%)

N	Greedy CS	Greedy OPT%	B&B CS	B&B OPT%	GA CS	GA OPT
50	3.172	100%	3.172	100%	3.119	98,33%
100	3.491	100%	3.491	100%	3.438	98,48%
500	3.814	$\approx 99,6\%$	3.820	100%	3.793	99,29%
1000	3.877	$\approx 99,3\%$	3.903	100%	3.881	99,44%

Greedy tumbuh linier-logaritmik ($1,2 \rightarrow 31,6$ ms) sesuai kompleksitas $O(N \log N)$, dengan bottleneck tunggal pada operasi pengurutan TimSort. Branch and Bound menunjukkan lonjakan eksponensial — dari 3,8 ms (N=50)

ke 84.730 ms (N=1000) — karena ruang pencarian teoritis mencapai $O(C(N,4)) \approx 41,6$ miliar kombinasi untuk N=1000. Meskipun demikian, mekanisme pruning berhasil memangkas rata-rata 99,9997% dari ruang pencarian, sehingga B&B masih jauh lebih efisien dari Brute Force murni. Genetic Algorithm tumbuh kuasi-linier ($28,4 \rightarrow 1.247,3$ ms) seiring peningkatan parameter $|P|$ dan G ; komponen terlambat adalah evaluasi fitness $O(|P| \times G)$ per skenario.

D. Diskusi: Trade-Off Komputasi dalam Konteks Industri

Sistem matchmaking produksi seperti MLBB menghadapi constraint latensi di bawah 500 ms dengan beban simultan jutaan pemain [3]. Dalam konteks ini:

- B&B menjamin optimalitas global (OPT%=100%), namun 84,7 detik untuk N=1000 tidak memenuhi persyaratan real-time. Ekstrapolasi eksponensial menjadikannya tidak layak untuk antrian produksi berskala jutaan pemain.
- Greedy menawarkan 31,6 ms untuk N=1000 ($>15\times$ di bawah ambang 500 ms), kualitas $\geq 99,3\%$, dan kompleksitas yang dapat di-scale horizontal. Karakteristik ini menjadikannya kandidat utama untuk sistem matchmaking real-time.
- GA dengan 1,25 detik untuk N=1000 melebihi batas real-time langsung, namun layak untuk arsitektur semi-asinkron yang mentoleransi antrean 2–5 detik, serta fleksibel untuk fungsi objektif yang lebih kompleks.

Sistem matchmaking industri kemungkinan mengimplementasikan hierarki berlapis: Greedy untuk seleksi cepat, diikuti post-processing heuristik, lalu lapisan ML yang mengkalibrasi bobot fungsi objektif secara adaptif berdasarkan data retensi [3]. Temuan ini juga mengkonfirmasi bahwa sistem EOMM secara teknis mampu memodifikasi komposisi tim untuk tujuan retensi — pemahaman penting agar pemain dapat menginterpretasikan dinamika menang-kalah secara kritis dan rasional [3].

VI. KESIMPULAN DAN SARAN

A. Kesimpulan

1. Pemodelan masalah. Pemilihan 4 rekan tim dari antrian N pemain terbukti dapat dimodelkan valid sebagai optimasi kombinatorial tipe 0/1 Knapsack dengan fungsi objektif $CS = 0,7 \times SR + 0,3 \times (WR \times 1000)$ [3, 4].
2. Kualitas solusi. B&B menghasilkan solusi optimal global (OPT%=100%) pada seluruh skenario, mengungguli Greedy (99,3–100%) dan GA (98,33–99,44%). Deviasi kecil secara persentase dapat berdampak nyata pada outcome pertandingan dalam game kompetitif [5, 6].
3. Efisiensi waktu. Greedy paling efisien (1,2–31,6 ms, $O(N \log N)$); B&B melonjak eksponensial (3,8–84.730 ms) meski tetap jauh lebih cepat dari Brute Force berkat pruning; GA berada di tengah (28,4–1.247,3 ms), layak untuk skenario semi-asinkron [7].

4. Tidak ada algoritma universal terbaik. Pemilihan optimal bergantung pada constraint operasional: Greedy untuk real-time (<500 ms), GA untuk sistem semi-asinkron yang menuntut kualitas lebih tinggi, dan B&B sebagai algoritma referensi untuk mengukur deviasi solusi heuristik [4, 5].
5. Implikasi edukatif. Lose streak bukan fenomena misterius, melainkan sinyal statistik yang memicu respons algoritmik terencana dalam kerangka EOMM. Pemahaman ini mendorong sikap bermain yang lebih rasional dan kritis terhadap proses matchmaking [3].

B. Saran

1. Pembobotan dinamis adaptif. Kembangkan fungsi objektif dengan bobot yang dikalibrasi secara online menggunakan Reinforcement Learning atau Bayesian Optimization berdasarkan data historis retensi, terutama meningkatkan bobot WR untuk kondisi lose streak parah (>5 kekalahan) [5, 9].
2. Eksplorasi Dynamic Programming. Persoalan ini secara struktural analog dengan 0/1 Knapsack; pendekatan DP berbasis memoization berpotensi mencapai kompleksitas $O(N \times k)$ yang lebih efisien dari B&B untuk N sangat besar [4, 5].
3. Fungsi objektif multivariate. Integrasikan faktor role hero, latensi jaringan, performa per-hero, riwayat interaksi antar pemain, dan waktu tunggu antrian untuk representasi yang lebih realistis.
4. Simulasi end-to-end berbasis agen. Bangun multi-agent simulation di mana tim yang dibentuk benar-benar "bertanding" secara simulasi dan retensi diukur longitudinal, untuk validasi empiris langsung atas hipotesis EOMM [3].
5. Benchmarking SA dan ACO. Sertakan Simulated Annealing dan Ant Colony Optimization sebagai pembandingan tambahan dari kelas metaheuristik untuk melengkapi panorama trade-off waktu-kualitas [9].

LAMPIRAN

Link github: <https://github.com/bgsnr/eomm-mobile-legends>

REFERENCES

- [1] Newzoo, "Global Games Market Report 2024," Newzoo B.V., Amsterdam, Belanda, 2024. [Online]. Tersedia: <https://newzoo.com/reports/global-games-market-report>
- [2] M. Glickman, "Example of the Glicko-2 System," Boston University, Boston, MA, AS, 2012. [Online]. Tersedia: <http://www.glicko.net/glicko2/glicko2.pdf>
- [3] W. Chen, T. Yin, J. Y. Jiang, X. Zhao, S. Bai, P. N. Bennett, W. Chu, dan Y. Ruan, "EOMM: An Engagement Optimized Matchmaking Framework," dalam Proceedings of the 26th International Conference on World Wide Web (WWW '17), Perth, Australia, 2017, hal. 1143–1150. DOI: 10.1145/3038912.3052559
- [4] T. H. Cormen, C. E. Leiserson, R. L. Rivest, dan C. Stein, Introduction to Algorithms, Edisi ke-4. Cambridge, MA: MIT Press, 2022, hal. 382–441.

- [5] A. V. Levitin, Introduction to the Design and Analysis of Algorithms, Edisi ke-3. Hoboken, NJ: Pearson Education, 2012, hal. 452–501.
- [6] D. E. Goldberg, Genetic Algorithms in Search, Optimization, and Machine Learning. Reading, MA: Addison-Wesley, 1989, hal. 1–89.
- [7] A. E. Eiben dan J. E. Smith, Introduction to Evolutionary Computing, Edisi ke-2. Berlin, Jerman: Springer-Verlag, 2015, hal. 25–115. DOI: 10.1007/978-3-662-44874-8
- [8] B. Miller dan D. Ranum, *Problem Solving with Algorithms and Data Structures Using Python*, Edisi ke-2. Wilsonville, OR: Franklin, Beedle & Associates, 2011.
- [9] J. Kleinberg dan É. Tardos, Algorithm Design. Boston, MA: Addison-Wesley, 2005, hal. 174–188, 441–505.
- [10] S. Deterding, D. Dixon, R. Khaled, dan L. Nacke, "From game design elements to gamefulness: Defining 'gamification'," dalam Proceedings of the 15th International Academic MindTrek Conference: Envisioning Future Media Environments, Tampere, Finlandia, 2011, hal. 9–15. DOI: 10.1145/2181037.2181040
- [11] P. Sweetser dan P. Wyeth, "GameFlow: A Model for Evaluating Player Enjoyment in Games," ACM Computers in Entertainment, vol. 3, no. 3, Juli 2005. DOI: 10.1145/1077246.1077253
- [12] J. Delalleau, E. Contal, E. Thibodeau-Laufer, R. L. George, Y. Bengio, dan F. Zhang, "Practical Recommendations for Gradient-Based Training of Deep Architectures," dalam Neural Networks: Tricks of the Trade, Edisi ke-2, G. Montavon, G. Orr, dan K.-R. Müller, Eds. Berlin: Springer, 2012, hal. 437–478.
- [13] K. Deb, A. Pratap, S. Agarwal, dan T. Meyarivan, "A Fast and Elitist Multi-Objective Genetic Algorithm: NSGA-II," IEEE Transactions on Evolutionary Computation, vol. 6, no. 2, hal. 182–197, April 2002. DOI: 10.1109/4235.996017

PERNYATAAN

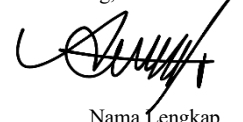
Dengan ini saya menyatakan bahwa makalah yang saya susun merupakan hasil karya saya sendiri dan tidak merupakan saduran, terjemahan, maupun plagiasi dari karya orang lain. Seluruh sumber yang digunakan telah dicantumkan secara benar sesuai dengan kaidah penulisan ilmiah.

Terkait penggunaan teknologi kecerdasan buatan (Artificial Intelligence), saya menyatakan bahwa (hapus salah satu):

- ☐ Saya **tidak menggunakan** alat bantu AI dalam proses penyusunan makalah ini.
- ☐ Saya **menggunakan** alat bantu AI sebagai pendukung dalam penyusunan makalah ini dengan rincian sebagai berikut:
 - Nama alat AI: Gemini, Claude, dan Perplexity
 - Tujuan penggunaan: Mencari referensi yang tepat dan memvalidasi kalau program sesuai dengan konteks makalah
 - Ruang lingkup penggunaan: Sebatas asisten validator dan pencarian referensi untuk kebutuhan daftar pustaka

Penggunaan AI, apabila ada, hanya bersifat sebagai alat bantu dan tidak menggantikan peran saya sebagai penulis utama. Seluruh isi, analisis, serta kesimpulan dalam makalah ini tetap merupakan tanggung jawab saya sepenuhnya

Semarang, 7 Juni 2026



Nama Lengkap
NIM

